Online Palengke — Search Button and Modal Practice

TESDA Web Development (Front-End) NC III | Companion to the Offline Assessment Handbook

This guide is a supplement to the main handbook. The original sample had a live search field, an inline Add/Edit form, and a browser-native window.confirm() delete prompt. This revised project includes a deliberate **Search** button, a **Clear** button, a popup **Add/Edit modal**, and a separate **Delete confirmation modal** in both the Vue example and the no-install HTML/CSS/JavaScript fallback.

> **Location of working code:** src/App.vue (Vue state, functions, template); src/style.css (CSS); offline-fallback/index.html (pure HTML/CSS/JavaScript implementation). You do not need to paste these lessons into the revised project because the features are already coded. Use the snippets to rebuild them yourself in a separate practice copy.

1. Why these topics matter

Search teaches input events, form submission, arrays, .filter(), strings, and reactive rendering. **Modals** teach component state, conditional HTML display, event handlers, forms, accessibility, and the distinction between a user interface decision and a backend operation.

Action	Trigger	Frontend result	Backend result
Type text	v-model="searchDraft"	Draft changes, cards do not yet change	No API call
Click Search or press Enter	@submit.prevent="submitSearch"	Cards filtered by the submitted query	No API call in this practice app
Click Clear	@click="clearSearch"	All loaded products displayed	No API call
Click Add product	@click="openCreate"	Popup opens with a blank form	No API call yet
Click Edit	@click="openEdit(product)"	Popup opens with product values	No API call yet
Click Save	@submit.prevent="saveProduct"	Validation, saving state, refreshed cards	POST or PATCH; then GET
Click Delete	@click="openDeleteConfirm(pro	Confirmation popup opens	No API call yet

Action	Trigger	Frontend result	Backend result
	duct)"		
Click Yes, delete	@click="confirmDelete"	Removes card only after success/reload	DELETE; then GET

2. Search button: exact code placement

Step A — src/App.vue, <script setup>, next to other ref() declarations

```
const query = ref('')      // The submitted search term
const searchDraft = ref('') // What the user is typing
```

There are two values so that typing alone does not filter until Search is submitted. ref('') creates reactive strings Vue tracks when they change.

Step B — src/App.vue, <script setup>, under your other computed values

```
const filteredProducts = computed(() =>
  products.value.filter(item =>
    String(item.name).toLowerCase().includes(query.value.toLowerCase().trim())
  )
)
```

- products.value is the entire array loaded from the server.
- .filter() creates a new array of matches without modifying the original.
- .includes() checks whether the product name contains the search query.
- .toLowerCase() means MANGO, mango, and Mango match equally.
- computed() recalculates when products or query changes.

Step C — src/App.vue, <script setup>, under the computed block

```
function submitSearch() {
  query.value = searchDraft.value.trim()
}
function clearSearch() {
  searchDraft.value = ''
  query.value = ''
}
```

Important: submitSearch() does not contact an API. It filters the products **already loaded** into Vue. If assessment instructions explicitly require server-side search, follow the provided API specification instead.

Step D — src/App.vue, <template>, inside the toolbar

```
<form class="search-form" role="search" @submit.prevent="submitSearch">
  <label class="search-label">
    <span class="sr-only">Search products</span>
```

```
    <input v-model="searchDraft" type="search"
        placeholder="Search by product name..." />
  </label>
  <button type="submit" class="secondary-button">Search</button>
  <button v-if="query" type="button" class="clear-button"
      @click="clearSearch">Clear</button>
</form>
```

The @submit.prevent event runs the same code for clicking the button **and pressing Enter**; .prevent stops the browser's normal page-reload form submission. v-if="query" shows Clear only after a search was submitted.

Step E — src/App.vue, <template>, where you render cards

```
<article v-for="product in filteredProducts" :key="product.id">
  <h3>{{ product.name }}</h3>
</article>
```

Do not loop over products if you expect the search to hide nonmatching cards. Use filteredProducts in v-for.

3. Add/Edit popup modal: exact placement

Step A — src/App.vue, <script setup>

```
const showForm = ref(false)
const editingId = ref(null)
const form = reactive({ name: '', description: '', price: 0, stock: 0 })
```

showForm decides whether the modal exists. editingId === null indicates **Add**. Any non-null ID indicates **Edit**. The reactive() object contains fields bound to the form inputs.

Step B — src/App.vue, <script setup>, define opener and closer

```
function openCreate() {
  editingId.value = null
  Object.assign(form, { name: '', description: '', price: 0, stock: 0 })
  showForm.value = true
}
function openEdit(product) {
  editingId.value = product.id
  Object.assign(form, {
    name: product.name,
    description: product.description ?? '',
    price: Number(product.price),
    stock: Number(product.stock)
  })
  showForm.value = true
```

```
}
function closeForm() {
  if (busy.value) return
  showForm.value = false
  editingId.value = null
}
```

The updated actual implementation also resets error messages and manages keyboard focus. This shorter snippet demonstrates the core logic. Object.assign() fills the same reactive object rather than replacing its reference.

Step C — src/App.vue, <template>, after toolbar and before the products list

```
<div v-if="showForm" class="modal-backdrop" @click.self="closeForm">
  <section class="modal-dialog" role="dialog" aria-modal="true"
           aria-labelledby="editor-title">
    <h2 id="editor-title">
      {{ editingId === null ? 'Add product' : 'Edit product' }}
    </h2>
    <form @submit.prevent="saveProduct">
      <label>Product name
        <input v-model.trim="form.name" required />
      </label>
      <label>Price
        <input v-model.number="form.price" type="number" min="0" required />
      </label>
      <label>Stock
        <input v-model.number="form.stock" type="number" min="0" required />
      </label>
      <button type="button" @click="closeForm">Cancel</button>
      <button type="submit" :disabled="busy">Save</button>
    </form>
  </section>
</div>
```

This is a **teaching fragment**, not a full replacement for the revised file's richer modal. @click.self closes the backdrop only if the user clicks the backdrop itself, not inside the form. Vue's v-model connects typed input values to the reactive form object.

Step D — src/style.css, append modal styling

```
.modal-backdrop {
  position: fixed;
  inset: 0;
  z-index: 1000;
  display: flex;
```

```
  align-items: center;
  justify-content: center;
  padding: 18px;
  background: rgba(7, 29, 17, .68);
}
.modal-dialog {
  width: min(640px, 100%);
  max-height: calc(100vh - 36px);
  overflow-y: auto;
  background: white;
  border-radius: 18px;
  padding: 25px;
}
```

position: fixed makes the dark backdrop cover the entire viewport. z-index keeps the popup above product cards. max-height and overflow-y make it usable on mobile devices.

Step E — Existing Save function: API versus UI

Your full project already has saveProduct(). The important structure is:

```
const payload = toPayload(form)
if (editingId.value === null) {
  await activeService.value.create(payload) // POST
} else {
  await activeService.value.update(editingId.value, payload) // PATCH
}
await loadProducts() // GET: refresh from the selected data source
```

Opening a modal does not write to the database. **Only Save triggers POST or PATCH.** If the server fails, keep the dialog open and display the error; do not pretend the product was saved.

4. Delete confirmation modal: where it belongs

Step A — src/App.vue, <script setup>

```
const showDeleteConfirm = ref(false)
const deletingProduct = ref(null)

function openDeleteConfirm(product) {
  deletingProduct.value = product
  showDeleteConfirm.value = true
}
function closeDeleteConfirm() {
  if (busy.value) return
  showDeleteConfirm.value = false
```

```
  deletingProduct.value = null
}
```

The Delete button **opens** a dialog instead of deleting immediately.

Step B — src/App.vue, <template>, after the Add/Edit modal

```
<div v-if="showDeleteConfirm" class="modal-backdrop">
  <section class="modal-dialog" role="dialog" aria-modal="true"
        aria-labelledby="delete-title">
    <h2 id="delete-title">Delete product?</h2>
    <p>Delete {{ deletingProduct?.name }}?</p>
    <button type="button" @click="closeDeleteConfirm">Cancel</button>
    <button type="button" :disabled="busy" @click="confirmDelete">
      Yes, delete
    </button>
  </section>
</div>
```

The updated App.vue contains the complete confirmDelete() function, error-handling behavior, and accessible modal interactions. It calls the service's DELETE operation and then loadProducts(); on failure the popup remains open so the error can be seen.

Step C — In each product card, replace the old Delete click handler

```
<button type="button" @click="openDeleteConfirm(product)">
  Delete
</button>
```

Remember: openDeleteConfirm(product) shows UI; confirmDelete() sends the DELETE request.

5. The exact workflow the assessor may ask you to explain

1. User clicks **Add product**, showForm becomes true, and v-if renders the modal.
2. User types; v-model stores the values in Vue's reactive form object.
3. Clicking **Save** triggers @submit.prevent="saveProduct".
4. Frontend validates input and sends a POST or PATCH request via productService.js (API mode).
5. When the server responds successfully, the frontend calls GET to reload current products.
6. Vue rerenders cards from the products array and closes the modal.
7. If POST/PATCH fails, error handling displays a message and leaves the form available for correction.
8. User clicks **Delete**; the confirmation modal opens, but nothing has been deleted yet.
9. User clicks **Yes, delete**; DELETE runs, followed by GET; if the API is unavailable, report the failure instead of silently removing data.

6. Accessibility and robustness checkpoints

The full Vue version adds role="dialog", aria-modal, a named heading, Escape-to-close, keyboard Tab navigation within the modal, and a return of focus to the previous control after closing. Buttons are disabled during save/delete operations to avoid repeated requests. The modal fits smaller screens and scrolls if needed. These measures improve usability, but they do not replace broader cross-browser and accessibility testing.

Browser-only search security: Vue interpolations ({{ product.name }}) escape ordinary text by default. Avoid blindly using v-html on untrusted product data.

Backend security: Client-side checks are bypassable. API authorization, input validation, safe database updates, CORS configuration, and authentication rules must be enforced by the server.

7. Practice assessment checklist

- ☐ Search for Tom and show Tomatoes; search is case insensitive.
- ☐ Verify typing alone does not filter; clicking **Search** or pressing **Enter** does.
- ☐ Click **Clear** and verify all loaded cards return.
- ☐ Add a product by modal; test required fields and invalid values.
- ☐ Edit a product using its popup; check the exact matching card updates.
- ☐ Open Delete confirmation, click Cancel, and verify nothing changes.
- ☐ Open again, click Yes, delete, and verify the API response and card refresh.
- ☐ Press Escape to close a modal and use Tab/Shift+Tab to navigate controls.
- ☐ Turn off the API and attempt Save/Delete in **Live API** mode: the operation must report failure, not silently succeed.
- ☐ Switch deliberately to **Offline demo** and explain why localStorage success is **not** an API integration test.
- ☐ Test at desktop, tablet, and 375px mobile viewport widths.
- ☐ Run npm test, npm run build, then repeat with Wi-Fi turned off after dependencies have been installed and backed up.

8. Common exam questions and short answers

What is a modal? A temporary interface overlay to complete a task or confirm a decision without navigating to another page.

What is v-if? A Vue directive that conditionally mounts or removes HTML according to a reactive expression.

What does @click.self mean? Run the click handler only if the event originated on the element itself, not a child inside it.

Does searching require an API? Not when filtering an already-loaded array on the client. For large datasets or requirements specifying server-side search, an API may be appropriate.

Does Cancel delete a product? No. Cancel only closes the confirmation dialog.

Why reload products after Save/Delete? To use the server's current returned data as the truth and avoid showing an unconfirmed change.

What if the API is unavailable? The browser should show an error. The deliberately chosen Offline demo uses localStorage only and is clearly labeled.

Assessment reminder: TESDA's competency criteria include HTML/CSS, JavaScript, a JavaScript framework, API access, testing, security, optimization, Git, and documentation. They do **not** guarantee that a specific assessment will require a modal or a search button. These are practical preparation exercises.